

Politecnico di Torino

Politecnico di Torino

Master of science program in CYBERSECURITY

A.a. 2025/2026

Graduation Session March 2026

An Experimental Framework for Evaluating Secure Deletion Techniques in Modern Digital Environments

Relatori:

Andrea Atzeni

Giuseppe Emiliano Vaciago

Candidati:

Imesha Ann Anjalie Ponnamperumage

Abstract

Acknowledgements

Table of Contents

List of Tables VI

List of Figures VII

1 Introduction 1
- 1.1 NAND flash storage architecture . . . 2
  - 1.1.1 Cell Structure and Memory Organization . . . 2
  - 1.1.2 Basic Operations and Timing . . . 3
  - 1.1.3 Write-Once Property and Out-of-Place Updates . . . 3
  - 1.1.4 Flash Translation Layer . . . 3
- 1.2 Implications for Secure Deletion . . . 4

2 Background and Related Work 5
- 2.1 A Taxonomy of Anti-Forensic Techniques . . . 6
- 2.2 Secure Deletion; Challenges and Limitations . . . 7
- 2.3 Traditional Secure Deletion Techniques . . . 8
- 2.4 The Flash Memory Challenge . . . 8
- 2.5 Secure Deletion in Virtualized and Containerized Environments . . . 9
- 2.6 Legal and Ethical Dimensions . . . 9
- 2.7 Summary and Identified Research Gaps . . . 9

3 Systematic Review of Secure Deletion Methods 10

4 Experimental Framework and Methodology 11

5 Experimental Results and Analysis 12

6 Interpretation of Findings and Implications 13

7 Conclusions and Future Work 14

A Appendix A 15

Bibliography 16

List of Tables

List of Figures

Chapter 1

Introduction

The task of deleting data and ensuring that they remain irrecoverable is known as Secure Data deletion. It has currently become one of the most pressing concerns in the cybersecurity, regulatory compliance, and digital forensics domains. While there exists physical means of making data irrecoverable in the physical world where confidential documents are shredded, and storage media containing classified material are physically destroyed, in the digital domain this is significantly complex. The standard delete operation on modern systems only implements file unlinking, which only changes the file system metadata while full contents will still remain intact in the system but users assume that all the content has been permanently erased. [1]

The criticality of this problem has been addressed by two intersecting forces. First, regulatory frameworks including GDPR right to erasure, NIST SP 800-88 Rev. 1, and IEEE 2883-2022 have mandated demonstrative and verifiable data removal evidences at defined sanitization levels; Clear, Purge and Destroy, with specific assurance against data recovery. Next, the mainstream transition from the usage of Hard Disk Drives (HDDs) to Solid-State Drives (SSDs) has led to a fundamental alteration of the storage landscape resulting in decades of secure deletion practices being inadequate.[2]

In contrast to HDDs, newer SSDs use a Flash Translation Layer (FTL) that maps logical block addresses to physical flash pages. The FTL performs out-of-place updates due to the erase-before-write property of NAND flash memory: modified data is written to a new physical location, and the old page containing the original data is marked as invalid. Furthermore, wear-leveling algorithms disperse data across the device to distribute program/erase cycles, and invalid pages are reclaimed by garbage-collection mechanisms only when free blocks run low, freeing them for new write operations. [2]

The problem of secure deletion not only occurs in the technical domain but also spans into the legal and ethical domains. On the legal front, regulations such

as the GDPR, the California Consumer Privacy Act (CCPA), and emerging AI governance frameworks impose obligations for data erasure that existing technical mechanisms struggle to satisfy.

The literature review below surveys state-of-the-art techniques in secure deletion, identifies methodological gaps, and motivates the need for a systematic framework that can reliably assess deletion effectiveness across diverse modern storage environments. The remainder of this thesis is organized as follows. Chapter 2 reviews the background of NAND flash storage architecture and the secure deletion problem in detail. Chapter 3 presents a systematic review of existing secure deletion techniques and their empirical evaluations. Chapter 4 describes the proposed experimental framework, including its metric system, test methodology, and verification protocols. Chapter 5 presents the results of applying the framework across a diverse set of storage devices and deletion techniques. Chapter 6 discusses the implications of the findings, Chapter 7 examines the legal and regulatory framework: the GDPR right to erasure, the right to be forgotten in emerging contexts, digital evidence and cross-border procedure, and broader data rights.and Chapter 8 concludes with directions for future work.

1.1 NAND flash storage architecture

NAND flash memory which was introduced by Toshiba in 1988, is a type of non-volatile semiconductor memory that has become the dominant storage technology for solid-state drives (SSDs), USB flash drives, and mobile devices [2]. Unlike NOR flash, which supports random access and high read performance at smaller capacities, NAND flash offers significantly higher storage density and faster write/erase operations at a lower cost, making it ideal for bulk data storage [3]

1.1.1 Cell Structure and Memory Organization

At the physical level, each NAND flash memory cell is a floating-gate transistor. Data is stored by injecting electrical charge into the floating gate, which is electrically isolated and can retain charge without power. A charged floating gate (programmed state) represents a logical "0," while an uncharged gate (erased state) represents a logical "1" [3]. Single-level cell (SLC) flash stores one bit per cell, whereas multi-level cell (MLC) stores two bits, triple-level cell (TLC) stores three bits, and quad-level cell (QLC) stores four bits. Each increase in bit density reduces cost per gigabyte at the expense of reduced endurance and lower performance [2].

Memory cells are arranged hierarchically. Cells connected to the same word line form a page, which is the smallest unit for read and program (write) operations. Pages are grouped into blocks, the smallest unit for erase operations. Blocks are organized into planes, multiple planes reside on a single die, and multiple dies are

packaged into a NAND chip. Typical page sizes range from 2 KB to 16 KB, and blocks typically contain 64 to 256 pages[2]. Each page also includes a spare or out-of-band (OOB) area used to store error-correcting codes (ECC) and metadata [4].

1.1.2 Basic Operations and Timing

The three fundamental operations on NAND flash are read, program, and erase. Read and program operate at page granularity, while erase operates at block granularity. Their latencies differ by orders of magnitude: a page read takes approximately 20 to 60 μs, a page program takes 200 to 500 μs μs, and a block erase takes 1.5 to 5 ms [2, 4]

1.1.3 Write-Once Property and Out-of-Place Updates

NAND flash has a fundamental constraint: a page cannot be overwritten in place. To write new data to a page, the entire encompassing block must first be erased, which resets all cells in that block to the "1" (erased) state [2] Because erasing a block is an expensive operation that destroys all data within it, SSDs do not perform in-place updates. Instead, they use out-of-place updates: modified data is written to an available free page elsewhere, and the old page is marked as invalid [5]. Pages containing current data are termed valid pages, while those holding superseded data are invalid pages. This write-once property is the root cause of many of the secure deletion challenges discussed in this review.

1.1.4 Flash Translation Layer

To maintain compatibility with traditional block-based file systems (e.g., ext4, NTFS), SSDs incorporate a Flash Translation Layer (FTL) that runs on the device controller [2]. The FTL maps logical block addresses (LBAs) issued by the operating system to physical flash addresses. This abstraction hides the complexity of NAND flash from the host and enables the SSD to manage internal operations transparently [4].

The FTL implements several critical management functions:

Wear leveling distributes program/erase (P/E) cycles evenly across all blocks to extend device lifespan. Each NAND flash block has a finite endurance typically 100,000 cycles for SLC and 10,000 to 100,000 for MLC/TLC after which the block becomes unreliable. Dynamic wear leveling distributes writes among free blocks, while static wear leveling periodically relocates infrequently accessed (cold) data to prevent some blocks from remaining idle while others wear out [3].

Garbage collection reclaims space occupied by invalid pages. When the number of free pages falls below a threshold, the FTL selects a victim block with a high proportion of invalid pages, copies any remaining valid pages to a free block, and then erases the victim block [2, 4]. This process introduces write amplification, where a single host write triggers multiple internal writes due to data migration.

Bad block management tracks blocks that have become defective through wear or manufacturing defects. These blocks are recorded in a bad block table and excluded from further use, with any valid data relocated to healthy blocks [4, 3].

Over-provisioning reserves a portion of the total flash capacity as spare area to improve performance and endurance. This over-provisioned space is used for wear leveling, garbage collection, and bad block management. [2, 6]

Error correction is essential because NAND flash cells are susceptible to read disturb, program disturb, charge leakage, and retention errors. The FTL computes ECC data stored in each page's OOB area to detect and correct bit errors[5].

An SSD packages multiple NAND chips with a controller that manages the FTL, host interface, and error correction. The storage architecture spans several layers: the host interface (e.g., SATA or NVMe) communicates with the FTL, which translates LBAs to physical addresses and coordinates access across multiple channels, packages, and dies in parallel to maximize throughput. To overcome the density scaling limits of planar (2D) NAND, manufacturers have adopted 3D NAND technology, which stacks memory cells vertically in layers [2]. 3D NAND achieves higher storage density without shrinking individual cell geometries, which improves endurance and reduces interference between adjacent cells. However, the vertical architecture introduces more complex disturb characteristics and new challenges for secure deletion, as erasing data in one layer can disturb adjacent layers [6].

1.2 Implications for Secure Deletion

The architectural features described above create fundamental obstacles to secure deletion. Out-of-place updates cause multiple copies of data to persist across the device as stale pages are left behind. Garbage collection and wear leveling can relocate data transparently, making it impossible for host-level software to predict where data physically resides. Over-provisioned areas are inaccessible to the operating system, yet may retain residual copies of deleted data. These characteristics mean that a conventional delete command at the file-system level does not remove the underlying data from the physical medium, and techniques that are effective on HDDs such as overwriting are ineffective on NAND flash [2].

Achieving secure deletion in modern systems is fundamentally challenging because storage architectures are organized into multiple layers application, file system,

block device, and physical medium each of which can retain residual copies of data [4]. Performing deletion at the application layer, for example, only removes the logical reference; the underlying data remains in the file system and may persist on the physical media indefinitely [5].

Chapter 2

Background and Related Work

The growing dependence on digital systems for both personal and professional activities has resulted in an unprecedented accumulation of sensitive information. As organizations and individuals increasingly store confidential data on a wide range of digital media—ranging from traditional hard drives to solid-state drives and cloud environments—the need for reliable and verifiable data deletion mechanisms has become critical. Failure to ensure proper data sanitization can expose users to privacy breaches, corporate espionage, and regulatory non-compliance.

From a cybersecurity perspective, secure deletion plays a dual role. On one hand, it is an essential component of data protection and privacy preservation, ensuring that information is irretrievably removed when no longer needed. On the other hand, it can also be employed as part of anti-forensic strategies, deliberately obstructing digital investigations by erasing potential evidence. Understanding this duality is fundamental to appreciating the technical and ethical challenges that surround secure deletion.

The following sections provide an overview of the conceptual and technical foundations that inform this research. First, a taxonomy of anti-forensic techniques is presented to situate secure deletion within the broader context of digital obfuscation strategies. Then, the principles of data deletion and persistence are examined, followed by an analysis of traditional and modern deletion approaches across different storage technologies and virtualized environments. Finally, the legal and ethical implications of secure deletion are discussed, highlighting the complex balance between privacy rights and forensic accountability.

This background establishes the theoretical basis for the experimental framework proposed in this thesis, which aims to evaluate and compare the effectiveness of secure deletion methods across contemporary computing infrastructures.

2.1 A Taxonomy of Anti-Forensic Techniques

The field of digital forensics is founded on the assumption that traces of user activity persist within digital systems and can be extracted for analysis and attribution. Anti-Forensics challenges this assumption by allowing the deliberate use of various techniques, procedures, tools and stratergies to obstruct the collection of digital evidences from a computer system. The main objective of these methodologies are to hinder Data Collection and impede Forensic investigations. These techniques aim to obscure, alter, or destroy digital evidence in ways that reduce the likelihood of recovery or interpretation. González Arias, et al. [7] provide one of the most comprehensive systematic reviews of this domain, analyzing 220 studies spanning from 2006 to 2022. Their taxonomy identifies four overarching categories of anti-forensic activity: **data destruction, artefact deletion, obfuscation, and attacks on forensic tools and methods.**

- **Data destruction** refers to the deliberate and irreversible removal of information, often achieved through overwriting, encryption, or physical destruction.
- **Artifact deletion** refers to the deliberate deletion of data to remove traces of activity—this is where *secure deletion* techniques become highly relevant.
- **Obfuscation** aims to mislead investigators disguising the nature, structure, or location of data to hinder recognition and analysis (e.g., steganography or data fragmentation).
- **Attacks of Forensic Tools and Methods** exploit vulnerabilities in forensic software or hardware to corrupt or mislead analysis.

This review highlights that while forensic techniques continue to evolve, so too do the methods employed to counteract them. But anti-forensic strategies remains a neglected research field, lacking standardized terminology, empirical testing, and benchmark datasets which makes it difficult to compare results across studies or build systematic defenses. Understanding these categories provides context for why secure deletion is often used for privacy or compliance, and also as an anti-forensic tool. However, this dual use highlights the ethical balance between data protection and evidence preservation that this research seeks to explore. Secure deletion has an unclear place in this classification. If it is used to hide criminal conduct, it could be considered a purposeful anti-forensic act, although it could fulfill lawful privacy or regulatory compliance aims, such as the EU's *right to erasure.* Context and intent make the difference, as Arias et al note, [7] "the same technique that safeguards confidentiality can, in adversarial hands, obstruct justice." Therefore, any balanced cybersecurity framework must take both forensic countermeasures and privacy-preserving computing into account when studying secure deletion.

2.2 Secure Deletion; Challenges and Limitations

Data deletion in digital systems is far more complex than its terminology suggests. Deleting a file from an operating system interface rarely removes its physical presence from the storage medium. Instead, the system merely updates metadata; marking sectors as available for reuse while leaving the original data intact until overwritten. Joukov, Papaxenopoulos, and Zadok [8] famously identified several pervasive misconceptions about secure deletion that persist even among professionals. Chief among these is the assumption that encryption alone guarantees data destruction, or that multiple overwrites are necessary for secure erasure. Their research demonstrated that a single, atomic overwrite operation is generally sufficient for magnetic storage, provided it is executed consistently and covers all data replicas, including journaling and metadata. However, the authors also highlighted the challenge of maintaining crash consistency — ensuring that deletion operations complete even in the event of system interruptions or power loss. This observation remains crucial in the context of modern SSDs, where caches and mapping tables may retain data even after deletion is logically completed.

Reardon, Basin, and Čapkun [1] contributed a foundational Systematization of Knowledge (SoK) on secure data deletion, bridging the gap between disparate methods and definitions. They proposed a layered taxonomy consisting of the user layer, file system, device driver, controller, and physical hardware. Each layer offers varying trade-offs between identifiability (the precision with which data can be targeted for deletion) and irrecoverability (the assurance that deleted data cannot be restored).

Their analysis revealed that higher layers (e.g., application or file system) possess semantic understanding of what to delete but lack physical control over the underlying media. Conversely, lower layers (e.g., hardware controllers) can permanently erase data but cannot discern its logical context. The tension between these layers defines the central challenge of secure deletion: achieving both semantic correctness and physical assurance.

Moreover, Reardon et al. [1] emphasized the importance of considering adversary models — differentiating between threats such as casual software recovery, forensic examination, or physical chip-level analysis. A deletion method effective against software recovery may still be vulnerable to forensic or hardware attacks. Thus, any secure deletion solution must be context-aware, calibrated to the adversary's capabilities and the sensitivity of the stored data.

2.3 Traditional Secure Deletion Techniques

Historically, secure deletion relied on *overwriting* — writing random or fixed patterns over the data to prevent recovery. Tools like shred, srm, and dd were widely used for this purpose. Standards such as the **U.S. Department of Defense (DoD 5220.22-M)** specified multiple overwrite passes to ensure data erasure on magnetic media.

However, as storage technologies evolved, it became clear that overwriting is not always reliable:

- Logical and physical mapping layers (e.g., in SSDs) can redirect writes, leaving original data intact.
- Wear leveling mechanisms in flash memory complicate direct access to physical storage locations.
- Modern drives may maintain hidden or reserved areas inaccessible to user-level commands.

While traditional techniques remain relevant for certain media, they no longer guarantee complete data sanitization on modern hardware.

2.4 The Flash Memory Challenge

Flash-based storage, including SSDs and USB drives, introduces unique challenges for secure deletion due to its internal **wear-leveling algorithms and block-based architecture**. Unlike magnetic disks, flash memory cannot overwrite data directly; instead, it must erase entire blocks before writing new data. This results in copies of deleted information potentially persisting in unaddressed cells.

Additional complications arise from:

- **Over-provisioning:** Manufacturers reserve hidden blocks for performance, inaccessible to standard overwriting commands.
- **Garbage collection:** Automated internal processes may relocate or temporarily preserve deleted data.
- **TRIM command behavior:** While intended to improve SSD performance, TRIM's effect on secure deletion is inconsistent across devices and operating systems.

These characteristics make flash memory a critical focus area for experimental evaluation frameworks like the one proposed in this thesis.

2.5 Secure Deletion in Virtualized and Containerized Environments

While secure deletion has been extensively studied in classical filesystems, flash storage, cloud storage, and virtual machines, there is virtually no focused research evaluating secure deletion inside containerized environments (e.g., Docker, containerd, Kubernetes ephemeral storage). This absence highlights the need for foundational work in defining threat models, measuring residual artifacts, and establishing deletion guarantees in layered container storage systems.

2.6 Legal and Ethical Dimensions

2.7 Summary and Identified Research Gaps

Chapter 3

Systematic Review of Secure Deletion Methods

Chapter 4

Experimental Framework and Methodology

Chapter 5

Experimental Results and Analysis

Chapter 6

Interpretation of Findings and Implications

Chapter 7

Conclusions and Future Work

Appendix A

Appendix A

```
1     const test = test;
```

$\mathcal{O}\left(n \log n\right)$
`numpy`

Bibliography

[1] Joel Reardon, David Basin, and Srdjan Čapkun. «SoK: Secure Data Deletion». In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP'13)*. San Francisco, CA, USA: IEEE, 2013, pp. 301–315. DOI: 10.1109/SP.2013.28 (cit. on pp. 1, 7).

[2] Fabian Aronsson and Oskar Lund. «Secure Data Deletion: Ensuring Confidentiality in Digital Systems ; A Survey of Methods for Data Erasure». LIU-IDA/LITH-EX-A–2025/074–SE. Master's thesis. Linköping, Sweden: Linköping University, 2025. URL: http://www.ep.liu.se/ (cit. on pp. 1–4).

[3] Omar A. Homaidi. «Data Remanence: Secure Deletion of Data in SSDs». Bachelor's thesis. Blekinge Institute of Technology, 2009. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-5165 (cit. on pp. 2–4).

[4] Niusen Chen and Bo Chen. «Duplicates also Matter! Towards Secure Deletion on Flash-based Storage Media by Removing Duplicates». In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. ASIA CCS '22. Nagasaki, Japan: Association for Computing Machinery, 2022, pp. 54–66. ISBN: 9781450391405. DOI: 10.1145/3488932.3523255. URL: https://doi.org/10.1145/3488932.3523255 (cit. on pp. 3, 4).

[5] Qi Zhang, Siqi Jia, Bo Chang, Yan Chen, and Zheli Liu. «Ensuring data confidentiality via plausibly deniable encryption and secure deletion – a survey». In: *Cybersecurity* 1.1 (2018), p. 1. DOI: 10.1186/s42400-018-0005-8. URL: https://doi.org/10.1186/s42400-018-0005-8 (cit. on pp. 3, 4).

[6] Na Young Ahn and Dong Hoon Lee. «Forensics and Anti-Forensics of a NAND Flash Memory: From a Copy-Back Program Perspective». In: *IEEE Access* 9 (2021), pp. 14130–14137. DOI: 10.1109/ACCESS.2021.3052353 (cit. on p. 4).

[7] Rafael González Arias, Javier Bermejo Higuera, J. Javier Rainer Granados, Juan Ramón Bermejo Higuera, and Juan Antonio Sicilia Montalvo. «Systematic Review: Anti-Forensic Computer Techniques». In: *Applied Sciences* 14.12 (2024). ISSN: 2076-3417. DOI: 10.3390/app14125302. URL: https://www.mdpi.com/2076-3417/14/12/5302 (cit. on p. 6).

[8] Nikolai Joukov, Harry Papaxenopoulos, and Erez Zadok. «Secure deletion myths, issues, and solutions». In: *Proceedings of the Second ACM Workshop on Storage Security and Survivability*. StorageSS '06. Alexandria, Virginia, USA: Association for Computing Machinery, 2006, pp. 61–66. ISBN: 1595935525. DOI: 10.1145/1179559.1179571. URL: https://doi.org/10.1145/1179559.1179571 (cit. on p. 7).